

고정점을 이용한 프로그래밍 언어 거리 측정

한상곤, 천준석, 우균
부산대학교 대학원 정보융합공학과
e-mail : {sangkon, jscheon, woogyuun}@pusan.ac.kr

Measuring the Distance of Programming Languages using Fixed-Points

Sangkon Han, Junseok Cheon, and Gyun Woo

1. 서론

기존의 프로그래밍 언어 유사도 평가는 구문이나 패러다임 등 정성적 기준을 활용하고 있어서, 실제 코드 변환 시 발생하는 구조적, 의미적 차이에 관심이 높다. 대규모 언어 모델(LLM: large language models)의 발전으로 이를 번역기처럼 활용해 언어 간 변형을 직접 측정하는 새로운 방법에 관한 논의가 활발하다. 본 연구는 자연어 처리의 고정점 평가 기법인 왕복 번역을 코드에 적용하면, 언어 간 거리를 정량적으로 도출할 수 있다는 가설에서 출발한다[1].

본 연구의 핵심 목적은 두 가지다. 첫째, C++를 기준 언어로 삼아 C, Java, Python을 오가는 자동화된 왕복 번역 파이프라인을 기반으로 한 RTT 기반의 언어 거리 측정 프레임워크를 제안한다. 둘째, 동일 조건에서 로컬 오픈소스 모델(qwen2.5-coder:7b)과 상용 모델(gpt-5.4)의 번역 양상을 비교하여, 모델의 고유한 특성이 언어 간 거리 측정 결과에 미치는 영향을 심층적으로 분석한다.

2. 관련 연구

2.1 왕복 번역 기반의 코드 의미 보존

왕복 번역(round-trip translation, RTT)은 최근 대규모 언어 모델의 환각 현상을 제어하고 코드 생성의 신뢰성을 평가하는 핵심 기법으로 자리 잡았다. 코드는 자연어와 달리 엄격한 구문과 실행 의미를 갖습니다. 최근 연구들은 정답 데이터셋이나 실행 환경 없이도 LLM의 코드 이해도를 평가하기 위해 왕복 정확도를 활용한다[2].

원본 코드 x 를 언어 A 에서 B 로 번역한 후 다시 A 로 복원했을 때, 복원된 코드 x' 이 원본 x 와 기능적으로 동일함을 검증하는 방식이다. 본 연구는 RTT 과정에서 필연적으로 발생하는 정보 손실(information loss)과 구조적 변형을 언어 간 거리(distance between languages)로 치환한다는 점에서 차별성을 갖는다. 번역을 반복할 때 의미가 보존되는 정도를 측정함으로써, 두 프로그래밍 언어가 공유하는 근본적 교집합의 크기를 정량화할 수 있다.

2.2 고정점(Fixed-point)과 수렴성

프로그램 분석학이나 컴파일러의 중간 표현(IR: intermediate representation) 최적화 과정에서는 변환 함수를 반복 적용했을 때 더 이상 코드가 변경되지 않는 상태, 즉 고정점 도달 여부를 중요하게 다룬다. LLM을 통한 코드 왕복 번역 $M^{-1}(M(x))$ 을 하나의 함수 적용 과정으로 본다면, 번역을 n 번 반복한 상태 x_n 은 결국 특정한 문체나 구조적 형태인

고정점 x^* 에 수렴하게 된다. 번역을 반복할 때 코드가 고정점 x^* 에 도달하기까지 걸리는 반복 횟수(iteration count)나, 원본 코드 x_0 와 수렴된 고정점 코드 x^* 사이의 형태적 차이는 두 언어 간의 구조적 이질성을 나타내는 훌륭한 척도이다. 예를 들어 C++와 C 사이의 RTT는 빠르게 고정점에 도달하지만, C++와 Python 사이의 RTT는 고정점에 도달하기까지 더 많은 구조적 왜곡을 겪거나 수렴하지 않고 진동할 수 있다. 이를 분석하면 LLM 모델의 번역 특성(qwen2.5-coder vs gpt-5.4)을 대조할 수 있다.

3. 실험 및 결과

3.1 실험 설계

실험 대상 코퍼스는 백준 온라인 저지(Baekjoon Online Judge)에서 선별한 5개 알고리즘 문제로 구성된다. 표 1은 실험 코퍼스에 대한 세부 사항이며, 특성에서 확인할 수 있듯이 서로 다른 알고리즘 패턴을 포함하도록 선택했다.

(표 1) 실험 코퍼스

문제 ID	문제명	특성
1436	영화감독 숏	문자열 탐색, 반복문
2110	공유기 설치	이분 탐색
2217	로프	정렬, 그리디 알고리즘
2579	계단 오르기	동적 프로그래밍
5567	결혼식	정렬, 구간 처리

기준 언어를 C++로 선택한 이유는 먼저, 세 대상 언어 모두와 구문적으로 어느 정도 공통 요소를 공유하기 때문이다. 둘째 이유로는, 각 언어로의 변환 시 서로 다른 수준의 구조적 변형이 필요하다는 점을 들 수 있다. C는 C++의 부분 집합에 가까운 구문 유사성을, Java는 객체지향의 필요성을, Python은 근본적으로 다른 구문 규칙(들여쓰기 기반, 동적 타입 시스템)을 각각 대표한다.

두 가지 LLM을 사용하여 동일 실험을 독립적으로 수행하였다. qwen2.5는 7B의 매개변수를 가지는 모델이며, 일반 컴퓨터에서 실행할 수 있는 수준이다. gpt-5.4는 정확한 매개변수가 알려져 있지 않으며, OpenAI의 클라우드 환경에서 API로 서비스가 제공된다. 두 모델 모두 온도(temperature)를 0으로 고정하여 결정론적(deterministic) 출력을 추구하였다. 다만, 온도를 0으로 설정하더라도 완전한 재현성은 보장되지 않

는다는 점에 주의해야 한다. 다만 여기서는 거리 측정의 목표에 맞게 최대한 결정론적 방향을 견지하였다.

각 실험 단위(문제-언어 쌍)에 대해 다음 지표는 다음과 같이 설정하였다.

- 변경 횟수 거리: 완료된 왕복 주기 수이며, 값이 클수록 수렴에 오래 걸림을 의미
- MOSS 유사도: Stanford MOSS 시스템을 활용한 코드 유사도 측정, 원본 C++ 과 왕복 후 C++을 비교
- 의미 보존 여부: 대상 언어 코드와 왕복 C++ 코드가 모든 샘플 입출력을 통과하는지 여부

3.2 실험 결과

표 2는 모델과 언어별 의미 보존 여부를 요약한 것으로, 표 2에서 성공은 최종 상태가 컴파일 및 실행할 수 있는 경우이며, 의미 통과는 대상 언어 코드가 실행되어 정답을 산출한 경우를 의미한다.

(표 2) 모델 및 언어 쌍에 따른 의미 보존 여부

언어 쌍	qwen2.5		gpt-5.4	
	성공	의미 통과	성공	의미 통과
C++ → C	2/5	3/5	5/5	5/5
C++ → Java	3/5	4/5	0/5	0/5
C++ → Python	3/5	3/5	0/5	0/5
전체	8/15	10/15	5/15	5/15

표 3은 언어별 평균 거리를 나타낸다. 해당 결과에서 주목할 만한 차이가 관찰된다. qwen2.5는 세 언어 쌍 모두에서 비교적 균일한 수렴 속도(평균 2.2~2.4회)를 보여 언어 쌍에 따른 편차가 작다. 반면 gpt-5.4는 C++ → C에서 평균 5.4회로 느린 수렴을 보이면서도, C++ → Java와 C++ → Python에서는 대부분 1회차에서 실패로 종료되어 평균값이 낮게 나타났다.

gpt-5.4에서 C++로부터 C로 변환할 때 발생하는 높은 반복 횟수는 역설적으로 해석할 수 있다. 모델이 번역 시 코드를 크게 재구성하면서도 의미적 정확성은 유지하여, 고정점에 도달하기까지 많은 조정이 필요했음을 시사한다.

(표 3) 언어 쌍별 평균 반복 횟수 및 변경 횟수 거리

언어 쌍	qwen2.5 평균	gpt-5.4 평균
C++ → C	2.2	5.4
C++ → Java	2.4	1.2
C++ → Python	2.2	1.0

표 4는 MOSS를 사용한 유사도 측정을 나타낸다. 하지만 gpt-5.4는 대부분의 문제와 언어 쌍에서 MOSS 점수가 20~50%대에 고르게 분포되어 있는 일관성을 보여주는 반면, qwen2.5는 0%가 나오거나 아예 코드 생성에 실패하는 경우가 잦지만, 성공한 경우는 70~80%에 육박하는 높은 점수 구조를 보여주는 양극화 현상을 보인다.

qwen2.5에서 파이썬으로 변환 후 다시 C++로 번역된 코드의 MOSS 수치를 보면 대부분 0%가 나오는데, 이는 구조적

의미로 해석할 수 있다. 즉, 역번역 과정에서 원본 C++ 프로그램의 구조적 흔적이 완전히 사라지는 형태로 재작성되었음을 의미한다. gpt-5.4는 파이썬으로 옮겼다 오더라도 30~60%대의 MOSS 유사도를 유지했다. 원본 문법 및 알고리즘의 뼈대를 최대한 보존하면서 번역하려는 경향이 더 강하다고 볼 수 있다.

(표 4) 문제별 MOSS 유사도

문제 ID	qwen2.5	gpt-5.4
1436	38.0%	28.7%
2110	28.7%	36.0%
2217	0.0%	20.7%
2579	76.0%	51.0%
5567	48.5%	35.7%

4. 제언

각 언어 쌍에 맞춰 최적의 모델을 선택하는 맞춤형 전략을 채택해야 한다. 예컨대, C++를 C로 변환할 때는 100% 성공률을 보인 gpt-5.4가 가장 적합하지만, 코드 구조가 크게 변형될 수 있다는 단점이 있다. 만약 구조 보존이 필수적이라면 더 작은 모델을 검토하는 것도 좋은 전략이다. 반면, C++를 Python으로 변환할 때에도 역시 qwen2.5가 성공률로 상대적 우위를 보인다. 하지만 실행시간 오류(run-time error)가 발생 위험이 존재하므로 주의가 필요하다.

5. 결론

본 연구의 실험 결과, qwen2.5 모델은 세 가지 언어 쌍 모두에서 언어 간 편차가 적은 균일한 수렴 속도(평균 2.2~2.4회)를 보였다. qwen2.5 모델은 전반적으로 범용적인 변환 능력을 입증했다. 반면, gpt-5.4 모델은 C++에서 C로의 변환 작업에서 완벽한 성공률을 달성한 것과 달리, Java와 Python 변환에서는 대부분 1회차에 조기 실패하는 극단적인 성능 차이를 보였다. 주목할 점은 gpt-5.4가 C 변환 시 기록한 높은 평균 반복 횟수(5.4회)로, 이는 모델이 코드를 크게 재구성하면서도 의미적 정확성을 잃지 않고 고정점에 도달하기까지 지속적인 조정 과정을 거쳤음을 시사한다. 또한, MOSS 기반의 코드 유사도 측정을 통해 두 모델이 구조를 다루는 방식에서 확연히 다른 번역 경향성을 띠고 있음을 확인했다.

향후 연구에서는 LLM이 원본 코드의 구조를 보존하려는 성향과 의미만 유지한 채 완전히 재작성하려는 성향을 보이는 근본적 원인을 심층 분석하여, 실무적 요구사항(구조 유지 vs. 단순 실행)에 맞춰 최적의 모델을 자동 할당하는 프레임워크 개발을 계속 진행할 계획이다.

참고 문헌

- [1] 조찬우, 김영훈, 우균, “왕복 번역과 고정점을 이용한 언어 거리 계산,” 2025 한국정보과학회 학술발표논문집, 2025.
- [2] Miltiadis Allamanis, Sheena Panthaplackel, Pengcheng Yin, “Unsupervised evaluation of code LLMs with round-trip correctness,” In Proceedings of the 41st International Conference on Machine Learning (ICML/24), Vol. 235, pp. 1050 - 1066, 2024.